Լաբորատոր աշխատանք 8
6 Ավտոկայանատեղի

Աշխատանքի նպատակը

Ստեղծել տվյալները և դրանց մշակման գործողությունները միավորող դաս ու կիրառել տարբեր պարամետրերով կոնստրուկտորներ:

Աշխատանքի պահանջը

Ստեղծել `ParkingSession` դասը: Պահպանվող տվյալներն են՝ մեքենայի համարանիշը (`string`), կայանման ժամերը (`int`), ժամային սակագինը (`double`) և անվճար ժամերի քանակը (`int`): Դասում ստեղծել երեք կոնստրուկտոր.

1. Առանց պարամետրերի կոնստրուկտոր, որը դաշտերին վերագրում է զրոյական/դատարկ արժեքներ:
2. Երեք պարամետրով կոնստրուկտոր, որը ստանում է համարանիշը, ժամերը, սակագինը (իսկ անվճար ժամերը դնում է 0):
3. Չորս պարամետրով կոնստրուկտոր, որը ստանում է բոլոր տվյալները:

Ստեղծել նաև՝

- `extendParking()` մեթոդը՝ կայանումը դրական ժամերով երկարացնելու համար,
- Հաշվարկային մեթոդներ (`getBillableHours()` և `calculateFee()`)՝ վճարովի ժամերը և ընդհանուր վճարը հաշվելու համար,
- `show()` մեթոդը՝ օբյեկտի տվյալները արտածելու համար:

Առաջադրանք

1. Ստեղծել `ParkingSession` դասը:
2. Իրականացնել երեք տարբեր կոնստրուկտորները:
3. Իրականացնել փոփոխող (`extendParking`) և հաշվարկային մեթոդները:
4. `main()` ֆունկցիայում ստեղծել օբյեկտ՝ օգտագործելով համապատասխան կոնստրուկտորը (ստուգիչ օրինակի տվյալներով):
5. Արտածել օբյեկտի տվյալները, վճարովի ժամերը և հաշվարկված ընդհանուր վճարը:

```
#include <iostream>

#include <string>

using namespace std;

class ParkingSession {

private:

    string carNumber;
```

```
    int totalHours;
    double hourlyRate;
    int freeHours;
public:
    // 1. Առանց պարամետրերի կոնստրուկտոր
    ParkingSession() {
        carNumber = "";
        totalHours = 0;
        hourlyRate = 0.0;
        freeHours = 0;    }
    // Երեք պարամետրով կոնստրուկտոր
    ParkingSession(string num, int hours, double rate) {
        carNumber = num;
        totalHours = (hours > 0) ? hours : 0;
        hourlyRate = (rate > 0) ? rate : 0.0;
        freeHours = 0;   }
    // Չորս պարամետրով կոնստրուկտոր
    ParkingSession(string num, int hours, double rate, int free) {
        carNumber = num;
        totalHours = (hours > 0) ? hours : 0;
        hourlyRate = (rate > 0) ? rate : 0.0;
        freeHours = (free >= 0 && free <= totalHours) ? free : 0;  }
    // 3. Փոփոխող մեթոդ՝ երկարացնել կայանումը դրական ժամերով
    void extendParking(int extraHours) {
        if (extraHours > 0) {
            totalHours += extraHours;
```

```
        }
    }
    // 4. Հաշվարկային մեթոդներ՝ հաշվել վճարովի ժամերը և վճարը
    int getBillableHours() const {
        int billable = totalHours - freeHours;
        return (billable > 0) ? billable : 0;
    }
    double calculateFee() const {
        return getBillableHours() * hourlyRate;
    }
    // Ցուցադրման մեթոդ
    void show() const {
        cout << "Car Number: " << carNumber << endl;
        cout << "Total Hours: " << totalHours << endl;
        cout << "Hourly Rate: " << hourlyRate << endl;
        cout << "Free Hours: " << freeHours << endl;
        cout << "Billable Hours: " << getBillableHours() << endl;
        cout << "Total Fee: " << calculateFee() << endl;    }
};
int main() {
    // 6. Ստուգիչ օրինակ. Մուտք՝ ժամեր 6, սակագին 300, անվճար ժամեր 2
    ParkingSession session("ABC-123", 6, 300.0, 2);
    cout << "===== Parking Session Details =====" << endl;
    session.show();
    return 0;
}
```